

Caregiver Communications Platform API

Complete CRUD API with multiple entities across 7 routers (auth, staff, locations, clients, shifts, documents, other).

Architecture Overview

Models vs Schemas

This API follows the **FastAPI + SQLAlchemy** pattern, separating database structure from API contracts:

Models (**models.py**) = Database Tables (SQLAlchemy ORM)

- **Purpose:** Define the PostgreSQL database schema
- **What they do:** Map Python classes to database tables
- **Usage:** Query and save data (`db.query(Staff)`, `db.add(new_client)`)
- **Contains:** All database columns, relationships, constraints
- **Example:** `Staff`, `Client`, `Shift`

```
# models.py - Database structure
class Staff(Base):
    __tablename__ = "staff"

    id = Column(Integer, primary_key=True)
    password_hash = Column(String(255)) # Stored in DB
    role_id = Column(Integer, ForeignKey("staff_roles.id"))

    role = relationship("StaffRole") # NOT a DB column, just ORM helper
```

Schemas (**schemas.py**) = API Contracts (Pydantic)

- **Purpose:** Define JSON request/response shapes for the API
- **What they do:** Validate input, serialize output, generate docs
- **Usage:** `response_model=StaffOut`, `data: StaffCreate`
- **Contains:** Only safe/relevant fields for each operation
- **Types:** `*Create` (POST), `*Update` (PUT), `*Out` (responses)

```
# schemas.py - API contracts
class StaffCreate(BaseModel): # Input - what clients send
    password: str # Plain text (before hashing)
    role_id: int

class StaffOut(BaseModel): # Output - what API returns
    id: int
    role_name: str # Computed from join, not in DB!
    # NO password_hash! (security)
```

Key Differences

Aspect	Models (SQLAlchemy)	Schemas (Pydantic)
Purpose	Database structure	API input/output
Where	<code>db.query(Model)</code>	<code>response_model=Schema</code>
Contains	All DB columns	Only safe fields
Security	Has <code>password_hash</code>	Never exposes passwords
Computed	Stores <code>role_id</code>	Returns <code>role_name</code>

Why Separate Them?

- 1. **Security:** Prevent accidental exposure of sensitive fields (`password_hash`, `ssn`)
- 2. **Flexibility:** API can return computed fields not in the database
- 3. **Validation:** Pydantic validates input before it touches the database
- 4. **Different shapes:** Create/Update/Output have different required fields

Quick Start

```
# Rebuild and restart server
docker-compose build server
docker-compose up -d

# View API docs
open https://localhost:8443/docs
```

API Structure

0. Auth Router (`/auth`)

Authentication:

- `POST /auth/login` - Authenticate user, returns JWT tokens
- `POST /auth/logout` - Log out (invalidates refresh token)
- `PUT /auth/change-password` - Change password (requires current password)
- `GET /auth/me` - Get current authenticated user info

1. Staff Router (`/staff`)

Staff:

- `GET /staff` - List all staff with role names
- `GET /staff/{id}` - Get specific staff member

- **POST /staff** - Create new staff (password hashed with bcrypt)
- **PUT /staff/{id}** - Update staff member
- **POST /staff/{id}/reset-password** - Reset staff password (admin only)
- **POST /staff/{id}/profile-image** - Upload staff profile image
- **DELETE /staff/{id}** - Deactivate staff member (soft-delete)

Staff Roles:

- **GET /staff/roles** - List all staff roles
- **GET /staff/roles/{id}** - Get specific role
- **POST /staff/roles** - Create new role
- **PUT /staff/roles/{id}** - Update role
- **DELETE /staff/roles/{id}** - Deactivate role

2. Locations Router (/locations)

Reference Data:

- **GET /locations/residence-types** - List residence types
- **GET /locations/relationship-types** - List relationship types

Addresses:

- **GET /locations/addresses** - List all addresses
- **GET /locations/addresses/{id}** - Get specific address
- **POST /locations/addresses** - Create address
- **PUT /locations/addresses/{id}** - Update address
- **DELETE /locations/addresses/{id}** - Delete address

Location Types:

- **GET /locations/types** - List location types
- **GET /locations/types/{id}** - Get specific location type
- **POST /locations/types** - Create location type
- **DELETE /locations/types/{id}** - Delete location type

Program Locations:

- **GET /locations/programs** - List all program locations
- **GET /locations/programs/{id}** - Get specific program
- **POST /locations/programs** - Create program location
- **PUT /locations/programs/{id}** - Update program location
- **DELETE /locations/programs/{id}** - Delete program location

3. Clients Router (/clients)

Reference Data:

- **GET /clients/behavior-types** - List all behavior types

Clients:

- GET /clients - List all clients
- GET /clients/{id} - Get specific client
- POST /clients - Create client
- PUT /clients/{id} - Update client
- DELETE /clients/{id} - Deactivate client (soft-delete)

Client Profile Image:

- POST /clients/{client_id}/profile-image - Upload profile image

Client Contacts:

- GET /clients/{client_id}/contacts - List client contacts
- GET /clients/contacts/{contact_id} - Get specific contact
- POST /clients/contacts - Add contact (body includes client_id)
- PUT /clients/contacts/{id} - Update contact
- DELETE /clients/contacts/{id} - Deactivate contact (soft-delete)

Client Residence:

- GET /clients/{client_id}/residence - Get current residence
- GET /clients/{client_id}/residence/history - Get full residence history
- POST /clients/residence - Create residence record (body includes client_id)
- PUT /clients/{client_id}/residence/{residence_id} - Update residence
- DELETE /clients/{client_id}/residence - End current residence

Program Enrollments:

- GET /clients/{client_id}/enrollments - List enrollments
- POST /clients/enrollments - Create enrollment (body includes client_id)
- PUT /clients/enrollments/{id} - Update enrollment
- DELETE /clients/enrollments/{id} - End enrollment

Client Updates:

- GET /clients/{client_id}/updates - List updates for client
- GET /clients/{client_id}/updates/count - Get unread count
- GET /clients/{client_id}/updates/{update_id} - Get specific update
- POST /clients/{client_id}/updates - Create update
- POST /clients/{client_id}/updates/{update_id}/read - Mark update as read
- DELETE /clients/{client_id}/updates/{update_id} - Archive update (soft-delete)

Client Documents:

- GET /clients/{client_id}/documents - List documents for client
- GET /clients/{client_id}/documents/category/{category_id} - List documents by category
- POST /clients/{client_id}/documents - Upload client document
- POST /clients/{client_id}/documents/{document_id}/review - Record document review

Client Behaviors:

- GET /clients/{client_id}/behaviors - List behavior tracking records
- GET /clients/{client_id}/behavior-configs - List behavior configs
- POST /clients/{client_id}/behavior-configs - Create behavior config
- DELETE /clients/{client_id}/behavior-configs/{config_id} - Deactivate config

4. Shifts Router (/shifts)**Reference Data:**

- GET /shifts/task-status-types - List task status types
- GET /shifts/dashboard-stats - Get dashboard statistics

Task Categories:

- GET /shifts/task-categories - List categories
- GET /shifts/task-categories/{id} - Get specific category
- POST /shifts/task-categories - Create category
- PUT /shifts/task-categories/{id} - Update category
- DELETE /shifts/task-categories/{id} - Deactivate category

Tasks:

- GET /shifts/tasks - List all tasks
- GET /shifts/tasks/{id} - Get specific task
- POST /shifts/tasks - Create task
- POST /shifts/custom-tasks - Create custom task for current shift
- PUT /shifts/tasks/{id} - Update task
- DELETE /shifts/tasks/{id} - Deactivate task

Shift Templates:

- GET /shifts/templates - List templates
- GET /shifts/templates/{id} - Get specific template
- POST /shifts/templates - Create template
- PUT /shifts/templates/{id} - Update template
- DELETE /shifts/templates/{id} - Archive template (soft-delete)

Shift Positions:

- GET /shifts/positions - List positions
- POST /shifts/positions - Create position
- PUT /shifts/positions/{id} - Update position
- DELETE /shifts/positions/{id} - Deactivate position

Shift Position Tasks:

- GET /shifts/position-tasks - List all position tasks
- GET /shifts/position-tasks/{id} - Get specific position task
- POST /shifts/position-tasks - Create position task

- **PUT** /shifts/position-tasks/{id} - Update position task
- **DELETE** /shifts/position-tasks/{id} - Deactivate position task

Log Categories:

- **GET** /shifts/log-categories - List log categories
- **GET** /shifts/log-categories/{id} - Get specific log category
- **POST** /shifts/log-categories - Create log category
- **PUT** /shifts/log-categories/{id} - Update log category
- **DELETE** /shifts/log-categories/{id} - Deactivate log category

Shifts:

- **GET** /shifts - List all shifts
- **GET** /shifts/current - Get current user's active shift
- **GET** /shifts/unread - List shifts with unread logs for current user
- **GET** /shifts/{id} - Get specific shift
- **POST** /shifts - Create shift directly
- **POST** /shifts/from-template - Create shifts from template (with optional weekly repetition)
- **PUT** /shifts/{id} - Update shift
- **DELETE** /shifts/{id} - Cancel shift (soft-delete via cancelled_at)
- **POST** /shifts/{shift_id}/end - End shift with completion data

Shift Assignments:

- **GET** /shifts/{shift_id}/assignments - List assignments for shift
- **GET** /shifts/{shift_id}/available-staff - List available staff for shift
- **GET** /shifts/templates/{template_id}/available-staff - List available staff for template
- **POST** /shifts/{shift_id}/assignments - Create assignment for shift
- **PUT** /shifts/assignments/{id} - Update assignment
- **DELETE** /shifts/assignments/{id} - End assignment (soft-delete via ended_at)

Shift Task Status:

- **GET** /shifts/{shift_id}/task-status - List task statuses for shift
- **GET** /shifts/task-status/{status_id}/history - Get status change history
- **PUT** /shifts/task-status/{id} - Update task status (auto-created; to "delete" set status_id=4)

Shift Daily Logs:

- **GET** /shifts/{shift_id}/logs - List daily logs for shift
- **GET** /shifts/location/{location_id}/logs - List logs for location
- **POST** /shifts/logs - Create log entry (immutable after creation)
- **DELETE** /shifts/logs/{id} - Delete log entry (Admin/Director/Site Director only)

Staff Log Read Status:

- **POST** /shifts/mark-read - Mark shift logs as read

5. Other Router (/) - Behaviors

Behavior Types:

- `GET /behavior-types` - List all behavior types
- `POST /behavior-types` - Create behavior type
- `PUT /behavior-types/{id}` - Update behavior type
- `DELETE /behavior-types/{id}` - Deactivate behavior type

Behavior Tracking:

- `GET /behavior-tracking` - List tracking records (with optional filters)
- `POST /behavior-tracking` - Create tracking record

6. Documents Router (`/documents`)**Document Categories:**

- `GET /documents/categories` - List categories with subcategories
- `POST /documents/categories` - Create category

Document Subcategories:

- `GET /documents/subcategories` - List subcategories
- `POST /documents/subcategories` - Create subcategory

Document Templates:

- `GET /documents/templates` - List templates
- `POST /documents/templates` - Create template
- `PUT /documents/templates/{id}` - Update template
- `DELETE /documents/templates/{id}` - Archive template (soft-delete)

Expired Documents:

- `GET /documents/expired/count` - Get count of expired documents
- `GET /documents/expired` - List expired documents

File Structure

```
server/
├── main.py           # FastAPI app with middleware and router includes
├── database.py       # SQLAlchemy engine, SessionLocal, get_db
dependency
├── models.py         # SQLAlchemy ORM models (database tables)
├── schemas.py        # Pydantic schemas (API validation/serialization)
├── routers/
│   ├── __init__.py   # Router module initialization
│   ├── auth.py       # Authentication endpoints
│   ├── staff.py      # Staff + staff roles CRUD endpoints
│   ├── locations.py  # Addresses, location types, program locations
│   └── clients.py    # Clients, contacts, residence, enrollments,
updates
└── shifts.py        # Tasks, templates, shifts, assignments, logs
```

```

└─ documents.py      # Document categories, templates, client
documents
└─ other.py          # Behavior types and tracking

```

models.py (SQLAlchemy - Database Layer)

- **ORM models** mapping to PostgreSQL tables
- Defines columns, types, constraints, foreign keys
- Contains `relationship()` for easy joins (not DB columns!)
- Used with `db.query(Model)` for database operations
- Examples: `Staff`, `Client`, `Shift`, `DocumentTemplate`

schemas.py (Pydantic - API Layer)

- **81+ Pydantic models** for request/response validation
- Three types per entity:
 - `*Create` - POST request body (no `id`, may have plain `password`)
 - `*Update` - PUT request body (all fields optional for partial updates)
 - `*Out` - API response (includes `id`, computed fields, excludes sensitive data)
- Provides automatic validation, type checking, and OpenAPI docs
- Examples: `StaffCreate`, `StaffUpdate`, `StaffOut`

Security Features

- **HTTPS only** with TLS certificates
- **CORS** restricted to localhost/127.0.0.1 (expand for production)
- **Password hashing** with bcrypt for staff accounts
- **HIPAA security headers**: HSTS, X-Content-Type-Options, X-Frame-Options
- **Trusted host middleware** restricts allowed origins
- **Response models** prevent accidental exposure of sensitive fields

Testing

Access interactive API docs at:

- **Swagger UI**: <https://localhost:8443/docs>

Notes

- All `PUT` endpoints use partial updates (only provided fields are changed)
- Nested routes use path parameters (e.g., `/shifts/{shift_id}/logs`)
- Query parameters available for filtering on some endpoints
- JSONB fields (logs payload) accept arbitrary JSON structures
- Foreign key constraints enforced at database level
- Shift daily logs are **immutable** after creation (healthcare documentation standard)
- Task statuses are auto-created; use `status_id=4` for soft-delete instead of DELETE